

SocioMart Functional Document

1. Project Overview

SocioMart is a shared residential-society marketplace connecting home chefs/sellers with neighbours/buyers.

1.1 The Problem It Solves

- Home chefs currently take orders through scattered WhatsApp messages — orders get missed, quantities sell out silently.
- Buyers have no single place to see which kitchens are open today and what is still available right now.
- Sellers have no record of daily revenue, pending payments, or which dish is most popular.

1.2 The Solution in One Line

One shared platform: sellers publish daily offerings in under a minute (manual form, saved Favourites, or paste a WhatsApp message and let the app parse it), buyers order with 3 taps, and both sides get live status tracking with automatic inventory.

1.3 Who Uses It (3 Roles)

| Role | What They Do | How They Log In | |---|---|---| | Buyer | Browse kitchens, add items to a cart, place orders (prepaid or cash-on-delivery), track status | Mobile number + OTP | | Seller | Create offerings, control inventory, mark sold-out, manage order pipeline, view earnings | Mobile number + OTP (seller-flagged account) | | Admin | Approve/verify new kitchens, monitor the whole platform | Admin console |

2. Buyer Final Experience

2.1 Login

Buyer opens the app and enters a 10-digit mobile number. The system sends an OTP. In demo mode, the OTP is fixed and displayed on-screen (no real SMS delivery). After successful OTP verification, the

buyer is logged in via HttpSession cookie.

2.2 Home — Live Kitchen Discovery

The buyer lands on a card grid of every approved kitchen in the society. Each card shows the kitchen's name, food description, rating and a live "available today" badge. A search box filters by name or dish instantly. Clicking a card fires a MENU_VIEW analytics event, which powers the seller dashboard's real "views today" metric.

2.3 All Kitchens — Shareable Buyer Route

The dedicated client-facing Buyer route is ``/#/kitchens``. This SPA hash route renders the "All Kitchens" page with tab filters (LIVE_NOW, TOMORROW, PREORDER, ALL) and calls ``GET /api/discovery/kitchens?tab=...``. The raw ``/kitchens`` path has backend/admin route ownership conflict and is not the final Buyer navigation path.

2.4 Kitchen Search

On Home and the All Kitchens page, buyers can search by kitchen name or dish name. The search filters the visible kitchen cards instantly client-side.

2.5 Kitchen Cards

Each kitchen card shows:

- Kitchen avatar/image (or emoji fallback)
- Kitchen name
- Heart button ( / ) for favouriting
- Short description
- Status pill (Taking orders, Pre-orders open, Tomorrow, Currently closed)
- Orderable item count
- Rating
- Previously ordered badge
- Preview of item names
- "View Kitchen →" action

2.6 Kitchen Detail

Clicking a kitchen card opens the public kitchen page (``/#/kitchen/{id}``). This page shows:

- Hero section with banner, avatar, identity, tags, social links (WhatsApp, Instagram), and Enquire button

- Service area information
- About section with Read more
- Kitchen Gallery
- "Available Today" section with live offerings
- "Pre-order" section with pre-order offerings
- "Coming Up" strip for upcoming scheduled dishes

No misleading kitchen-level "Orders Open" status is shown. Availability is item-level.

2.7 Live Offerings & Item-Level Availability

Each offering card shows:

- Item name, description, price, unit
- Remaining quantity / sold-out state
- Cutoff time
- Ready by time
- Rating
- Add to cart / order action
- Fulfillment badge (live, pre-order, flexible pre-order)

Items that are sold out or past their order cutoff are visibly disabled. The server enforces the same rule again at order time.

2.8 Cutoff Handling

Each offering has its own cutoff time (HH:mm). Ordering after cutoff is rejected server-side with HTTP 409.

2.9 Cart

The cart bar (`#viewOrderBar`) shows:

- Item count
- Total price
- "View Order" action

The cart is single-kitchen only. Adding from a different kitchen prompts a confirmation modal.

2.10 Draft Order

When the buyer taps "Place Order" in the cart, the app calls `POST /api/buyer/orders/draft`. The backend:

- Re-prices the cart server-side
- Validates quantities against remaining stock
- Validates cutoff times
- Validates sold-out status
- Creates a draft order

2.11 Buyer Details

Buyer details (name, flat number, mobile) are confirmed from the logged-in profile. The Confirm Order page shows the delivery address pulled from the buyer's profile.

2.12 Order Summary

The Order Summary screen shows:

- Kitchen name and order number
- List of items with quantities and prices
- Item total
- Note to kitchen textarea
- "Place Order ✓ — ₹X →" sticky footer CTA

2.13 Confirm Order

The Confirm Order screen reviews the real backend draft. It shows:

- Kitchen name and order number
- Items with quantity and prices
- Order total
- Delivery information
- Deliver-to address from buyer profile
- Note to kitchen (if any)
- Payment-status selection (Paid / Will Pay Later)
- "Place Order ✓ — ₹X" sticky footer CTA

2.14 Payment-Status Selection

The buyer chooses one of two options:

- **Paid** — order is recorded as PAID and CONFIRMED
- **Will Pay Later** — order is recorded as WILL_PAY_LATER and ORDERED

There is NO real payment gateway. There is NO card processing, CVV processing, UPI gateway, bank transaction, payment capture, or fake transaction processing.

2.15 Place Order

Tapping "Place Order" calls `POST /api/buyer/orders/place` with the selected payment status. The backend:

- Atomically decrements inventory
- Validates stock availability
- Prevents duplicate orders
- Records the order with the chosen payment status

2.16 Buyer Orders

The Orders screen shows all orders grouped by status with filters (All, Active, Completed, Cancelled). Each order card shows:

- Order number
- Kitchen name
- Order date
- Order status pill
- Payment status badge (Paid, Will Pay Later, Pending, Cancelled)
- Item lines
- Total amount

2.17 Order Details

Tapping an order opens the order detail page showing full item breakdown, quantities, prices, and status.

2.18 Enquiries

Buyers can send enquiries to any kitchen via the "Enquire" button. Enquiries appear in the Orders & Enquiries screen under the Enquiries tab.

2.19 Favourites

Buyer Favourite Kitchens are separate from Seller Favourite Templates. Buyer Favourite Kitchens:

- Maximum 3 favourite kitchens
- Add using the heart button ( → )

- Filled heart (❤️) removes the favourite — clicking the filled heart unfavourites/removes it
- Removed kitchen disappears from Favourite Kitchens immediately and after refresh/reload
- After removal, the slot becomes reusable — buyer can favourite another kitchen
- Fourth favourite is prevented server-side with the message: "You can favourite up to 3 kitchens only."
- V1 does NOT include Favourite Food Items
- State remains consistent across All Kitchens, Favourite Kitchens, and Kitchen detail
- Duplicate favourite clicks are safe — backend enforces one favourite per buyer+kitchen

The Favourites screen shows saved kitchens with quick links to their pages. A horizontal chip row on Home and Food Hub shows up to 6 favourite kitchens. Frontend favourite implementation:

- `toggle-fav-kitchen` data-action on heart buttons
- `FAV\_CACHE` (client-side Set) keeps heart states in sync across all views
- `loadFavSet()` loads favourites from `/api/favourites` on navigation
- Removal updates the cache and re-renders the Favourites list immediately

2.20 Profile

The Profile screen shows the buyer's name, mobile, flat, society, and building. Buyers can edit their profile details.

2.21 Logout / Session

Buyers can log out via the Profile screen. Session timeout is 30 minutes. Cookies use SameSite=Lax.

3. Kitchen Availability

The kitchen marketplace shows live/active kitchens and offerings.

- A kitchen should be publicly useful only when it has active/live offerings.
- Expired, unavailable, closed or sold-out offerings are not orderable.
- Availability is enforced server-side.
- Availability is primarily item-level.
- There is NO misleading kitchen-level "Orders Open" status.

4. Aarti Kitchen Demo

The verified Aarti Kitchen demo includes:

- Kitchen image: `https://example.com/aarti-kitchen.jpg`

- Instagram support/link: `https://instagram.com/aartikitchen`
- Society/area information: Sunshine Society, Building B
- Live offerings: Poha, Modak (unlimited), Idli, Misal Pav, Puran Poli, Sabudana Khichdi, Thalipeeth
- Item-level cutoffs
- Pre-order offering

Verified preorder example:

- **Puran Poli (Pre-order)**
- isPreorder=true
- preorderType=FIXED
- future Monday date
- cutoff 12:00
- quantity 30
- price ₹70

5. Service Area

Buyer-facing wording on the kitchen detail page: "Orders are currently limited to [Area] and may be limited to selected societies." The area comes from the seller onboarding/profile-selected area (`k.society`). It is not a second manually entered duplicate field.

6. Ordering Flow

The exact final flow: 1. **Add to cart** — Buyer picks quantity per item. Quantity is capped at remaining stock. 2. **Draft order** — `POST /api/buyer/orders/draft` prices the cart. Server recomputes price from DB — client cannot tamper. 3. **Buyer details** — Name, flat number, mobile confirmed from profile. Validated and stored on the order. 4. **Order Summary** — Review items, add note to kitchen. 5. **Confirm Order** — Review full draft, choose payment status (Paid / Will Pay Later). 6. **Payment-status selection** — UI-only selection of Paid or Will Pay Later. No real payment gateway involved. 7. **Place order** — `POST /api/buyer/orders/place`. Inventory decremented atomically; sold-out rejected with HTTP 409. 8. **Buyer Orders** — Order shows in the buyer's order list with status tracking. 9. **Order status tracking** — Status transitions: PENDING → CONFIRMED → READY → DELIVERED → COMPLETED (or CANCELLED). Server guarantees:

- Server-side pricing — totals recomputed from DB, client totals ignored
- Quantity validation — $qty \leq \text{remaining stock}$
- Cutoff validation — ordering after cutoff rejected
- Sold-out validation — draft and place reject sold-out items

- Atomic inventory decrement
- Duplicate-order protection — same buyer+kitchen+details within window rejected HTTP 400
- Authorization — buyer can only access own orders

7. Historical Order Price Immutability

Once an order is placed, its transaction-time unit price, quantity, line total, and order total are immutable.

- Changing the current offering/catalogue price does NOT modify any historical order.
- Historical orders retain the exact price that existed when each order was created.
- New orders always use the current catalogue price at the time of order creation.
- Admin GMV/order value is calculated from persisted historical order totals, not from current offering prices.
- Seller historical order views use the order's transaction snapshot.
- Buyer historical order views use the order's transaction snapshot.

Example:

- Offering price = ₹20 → Buyer orders 100 units → Order total = ₹2,000
- Seller changes offering price to ₹30
- Historical order remains ₹2,000
- New order at ₹30 × 100 = ₹3,000
- Admin GMV for these two orders = ₹5,000

8. Payment — Final V1 Model

Final V1 payment model is **PAYMENT STATUS RECORDING ONLY**. Supported statuses:

- PENDING
- PAID
- WILL_PAY_LATER

Buyer can choose:

- **PAID** — order recorded as PAID and CONFIRMED
- **WILL PAY LATER** — order recorded as WILL_PAY_LATER and ORDERED

There is NO real payment gateway. There is NO:

- card processing
- CVV processing
- UPI gateway

- bank transaction
- payment capture
- fake transaction processing

OrderService.consumeOrder mapping:

- PENDING → CONFIRMED (legacy path)
- PAID → CONFIRMED
- WILL_PAY_LATER → ORDERED

8. Final Order Summary UI Fix

File: ``static/js/common.js`` ``updateCartBar()`` now hides ``#viewOrderBar`` when:

- cart has zero items

OR

- current view contains ``sticky-footer-bar``

This prevents duplicate/overlapping Place Order CTAs. Affected screens:

- Order Summary
- Confirm Order
- Payment

Expected result: Exactly ONE Place Order CTA. Other screens continue using the cart bar where appropriate. This was a frontend-only UI fix. No backend/business logic was changed.

9. Seller Final Experience

9.1 Seller Login

Seller logs in with mobile number + OTP. Seller-flagged accounts get seller role.

9.2 Dashboard

The seller dashboard answers four questions at a glance:

- How many people viewed my menu today? (real analytics count)
- What's still in stock?
- How much have I earned? (split into confirmed-today, pending and this-month)
- What orders need action?

9.3 Live Menu Views Analytics

Real MENU_VIEW events are logged and counted. The dashboard shows "Views Today" based on real backend data — no hardcoded numbers.

9.4 Live Inventory

Each live offering shows booked quantity and remaining availability. Unlimited items show "No limit".

9.5 Offerings

Sellers can create offerings via three methods:

- Manual form
- Favourite Templates
- WhatsApp Quick Post

9.6 Manual Offering Creation

Fill in name, description, price, unit, availability (today/tomorrow), order window start/end, quantity available, and optionally save as template.

9.7 Saved Favourite Templates

Seller Favourite Templates are separate from Buyer Favourite Kitchens.

- Maximum 3 seller templates
- Template save toggle in the Create Offering form: "Save as template (max 3)"
- Templates appear as pills on the Add Offering screen: "Quickly post from saved templates (max 3)"
- Clicking a template pre-fills the creation form

9.8 Template Publishing

Publishing a template creates a brand-new independent Product. Live edits to the offering never touch the saved template.

9.9 Template Independence

Each published template becomes a separate Product record. The template remains unchanged and can be republished again.

9.10 WhatsApp Quick Post

Paste a real WhatsApp message; the parser extracts name, price, quantity and ready-by time. The parser is read-only — nothing is published without the seller confirming.

9.11 Sold-Out

Sellers can force-mark any offering as sold out, including unlimited-quantity items.

9.12 Unlimited Quantity

Items with no max quantity have no stepper and show "No limit". They can still be force-marked sold out.

9.13 Orders

Seller sees all orders for their kitchen with date tabs (Today, Tomorrow, Pick date). Daily summary shows total orders, paid/pending/cancelled breakdown, and revenue.

9.14 Order Status Pipeline

Orders move through: PENDING → CONFIRMED → READY → DELIVERED → COMPLETED (or CANCELLED). Seller can update status transitions including cancel.

9.15 Daily Summary

Per-dish plates + revenue. Clicking a dish drills into Item Details showing each buyer row with name, quantity, mobile, and payment status.

9.16 Per-Dish Revenue

Revenue is calculated per product. Cancelled orders are excluded from plates, revenue, and earnings.

9.17 Buyer Drill-Down

From the daily summary, seller can view per-product customer details: buyer name, quantity, mobile number, order number, and payment status.

9.18 Earnings

Earnings screen shows:

- Confirmed Today

- Pending
- This Month
- Per-item breakdown with confirmed and pending revenue

9.19 Enquiries

Seller can view and respond to enquiries from buyers.

9.20 Profile

Seller can manage kitchen details: name, society, building, speciality, description, WhatsApp, Instagram, UPI ID.

10. Inventory

- Finite quantity: items have a max_quantity and remaining_quantity
- Remaining quantity: shown live on the kitchen page and seller dashboard
- +/- stepper: seller adjusts stock with delta-based updates
- Delta-based server update: `PATCH /api/seller-app/products/{id}/inventory` with a delta
- Race-safe/server-side arithmetic: stock updates use atomic database operations
- No negative inventory: stock cannot go below zero (HTTP 400)
- Sold-out: seller can force-mark sold out
- Unlimited quantity: items with no max quantity have no stepper, show "No limit", but can still be force-marked sold out
- Cross-seller ownership protection: seller B calling the endpoint on seller A's item gets HTTP 403

11. Admin Final Scope

11.1 Admin Console

The admin console is a separate single-page view (`admin.html`) hitting `/api/admin/\*` endpoints. It requires ADMIN or SUPER_ADMIN role.

11.2 Admin Features

- Dashboard with total orders, paid/will-pay-later breakdown, buyer/seller counts
- Buyers management (list all buyers)

- Sellers management (list all sellers, pending approvals, approve/reject/suspend)
- Kitchens management (list all kitchens, approve/reject/suspend)
- Offerings management (browse all product listings)
- Orders monitoring (full order list with status and payment)
- Enquiries management (customer inquiries)
- Analytics (platform metrics)
- Seller approval/verification workflow

11.3 Final Verification

- Dashboard values are based on real backend data
- No hardcoded authoritative dashboard numbers
- Counts reconcile with underlying records
- Paid and Will Pay Later counts/values are tracked
- Order relationships are ID-based
- Admin can see shared order data

12. Shared Database

Buyer, Seller and Admin use the **SAME** Spring Boot backend/runtime. Database:

`jdbc:h2:mem:sociomartdb` There are NOT separate Buyer/Seller/Admin databases. Verified result: Seller and Admin see the same order data. Do not claim frontend localStorage is the authoritative business database.

13. Security

Final security model with three roles: BUYER, SELLER, ADMIN. Verified boundaries:

- anonymous → Admin rejected (HTTP 401)
- Buyer → Admin rejected
- Seller → Admin rejected
- Buyer → seller orders rejected (HTTP 403/404)
- Buyer → another buyer's order rejected (HTTP 404)
- Seller cross-owner access rejected (HTTP 403)

Security features:

- Role-based access
- Ownership checks on every mutation

- ID isolation — buyers cannot access another buyer's orders
- Session timeout: 30 minutes
- SameSite=Lax cookie protection
- Custom UserSessionAuthorizationFilter

14. Business Validations

All final verified rules enforced server-side: | Rule | Enforcement | Verified By | |---|---|---| | Order cutoff time | Strict 24-hour HH:mm; invalid (e.g. 25:99) rejected HTTP 400; ordering after cutoff rejected | E2E | | Stock can't go negative | Delta pushing stock below 0 -> HTTP 400 | E2E | | Sold-out unorderable | Draft & place reject sold-out items (HTTP 409) | probe | | Seller template cap = 3 | 4th save -> HTTP 409 "Maximum 3 favourite templates allowed" | E2E | | Template independence | Publishing creates a new Product — live edits never touch the saved favourite | E2E | | Seller ownership | Every mutation checks kitchen owner; else HTTP 403 | E2E | | Unlimited quantity | No stepper; delta -> "no quantity limit"; sold-out still allowed | E2E | | Quick Post never auto-publishes | Parse is read-only — offerings unchanged after parse | E2E | | Cancelled != revenue | Excluded from plates, revenue, earnings, drill-down | E2E | | Earnings states | pending = PENDING only; confirmedToday = CONFIRMED/DELIVERED | E2E | | Server-side pricing | Totals recomputed from DB — client totals ignored | E2E | | Duplicate order prevention | Same buyer+kitchen+details within window rejected HTTP 400 | Live test | | Shared DB across roles | Seller and Admin see identical order data | Live test | | Buyer favourite cap = 3 | 4th favourite -> HTTP 409 "You can favourite up to 3 kitchens only" | Unit test |

15. Testing

15.1 Unit Tests (JUnit 5)

13/13 unit tests PASS. Classes:

- **OrderServicePaymentTest** — 5 tests: payment status → order status mapping
- **FavouriteServiceLimitTest** — 4 tests: 3-kitchen favourite cap enforcement
- **OrderServiceValidationTest** — 4 tests: cutoff, stock, duplicate order validation

15.2 E2E Verification

Major verified categories:

- Buyer flow (login, browse, cart, order, track)
- Seller flow (login, dashboard, offerings, inventory, orders, earnings)
- Admin flow (approval, monitoring, analytics)

- Shared DB (seller and admin see same data)
- Security (role boundaries, ownership checks, ID isolation)
- Duplicate orders
- Inventory integrity (atomic decrement, no negative stock)
- Payment statuses (PAID → CONFIRMED, WILL_PAY_LATER → ORDERED)
- Cutoff validation
- Favourites (buyer cap = 3)
- Quick Post (read-only parser)
- Admin reconciliation (counts match underlying records)

16. Final Release Verification

Final release verification covered:

- Favourite Kitchen add/remove fix (backend persistence + frontend FAV_CACHE sync)
- Favourite max-3 enforcement and slot reuse
- UI fix (duplicate Place Order CTA — cart bar hidden on sticky-footer screens)
- Tests (13/13 unit tests pass)
- Compile (mvnw clean compile)
- Package (mvnw package -DskipTests)
- Buyer regression
- Seller regression
- Admin regression
- Shared DB
- Data integrity
- Security
- Duplicate-order protection
- Inventory integrity
- H2 reset behaviour
- Secret/repository scan
- Documentation (MD + PDF)
- Client demo runbook
- Render deployment

Final status: **NO RELEASE BLOCKERS. RELEASE FROZEN — NO FURTHER CODE CHANGES REQUIRED.**

17. Git Release State

Final verified release commit: ``c3c6496af32227a923a26fcf08a0e8bd32c7b53c``

- HEAD == origin/main: YES
- Working tree: clean except ``.kilo/`` agent tooling, which is not committed
- Do not commit ``.kilo/``

18. Deployment

18.1 Public URLs (single Render service)

- **Buyer:** <https://sociomart-demo.onrender.com/>
- **Seller:** <https://sociomart-demo.onrender.com/seller.html>
- **Admin:** <https://sociomart-demo.onrender.com/admin.html>
- **API health probe:** <https://sociomart-demo.onrender.com/api/kitchens>

18.2 Technical Details

- Hosting: Render.com free Docker web service
- Single service serves Buyer (`/`), Seller (`/seller.html`), and Admin (`/admin.html`)
- Docker multi-stage build: Maven (build) → JRE 21 (runtime)
- Render auto-deploys on every push to main
- Render injects PORT environment variable; app binds to ``${PORT:8081}``
- Health check path: ``/api/kitchens``
- Free tier sleeps after ~15 minutes of inactivity
- Cold start on Render free tier can cause an initial loading/waking delay — this is expected demo behaviour

19. Demo Reset

H2 is in-memory by default (``jdbc:h2:mem:sociomartdb``). Therefore:

- Custom runtime data disappears after restart
- DemoDataSeeder reseeds demo data on every boot (idempotent, guarded by platform_settings flags)
- This is an accepted V1/demo limitation

20. OTP Limitation

No real OTP delivery in demo. Dev mode uses fixed OTP behaviour. Do not describe it as production-grade SMS OTP.